МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Google Form Quiz Solver Bot

Требования

Санкт-Петербург, 2026

Содержание

1 Введение **4**
1.1 Основные функции 4

2 Общие требования **5**
2.1 Управление версиями и сотрудничество 5
2.2 Целевые артефакты 5

3 Функциональные требования **6**
3.1 Аутентификация пользователей 6
3.2 Решение форм 6
3.2.1 Формат ввода 6
3.3 История и управление формами 7
3.4 Статус и информация 8
3.5 Администрирование (роль ADMIN) 8
3.6 Ограничения и лимиты 8
3.7 Источник данных для LLM 9

4 Общие требования **10**
4.1 Производительность 10
4.2 Надежность 10
4.3 Безопасность 10
4.4 Роли пользователей 10
4.5 Авторизация 11
4.6 Эндпоинты 11
4.6.1 Эндпоинт healthcheck 11
4.6.2 Эндпоинт users 11

5 Требования к технологическому стеку **12**

6 Требования к документации **13**

7 Требования к тестированию **14**

8 Руководство по сдаче проекта **15**
8.1 Требования 15
8.2 Архитектура 15
8.3 Полный проект 15

9 Глоссарий **16**

1 Введение

1.1 Основные функции

1. **Решение Google-форм**
   - Приём ссылки на форму через команду `/solve` или текстовое сообщение.
   - Извлечение ссылки с помощью регулярного выражения.
   - Генерация ответов через GigaChat (3 попытки при ошибках, таймаут 30 секунд).
   - Вывод списка вопросов с ответами и уровнями уверенности.
2. **История и управление формами**
   - Просмотр истории решений (`/history [day|week|month|all]`).
   - Автоматическое сохранение успешно решённых форм.
   - Просмотр сохранённых форм (`/forms`).
   - Удаление сохранённой формы (`/remove_form <id>`).
   - Повторное решение (`/rescore <id>`).
3. **Статус запросов**
   - Проверка состояния обработки запроса (`/status <requestId>`) с возможными статусами: *Ожидает*, *В обработке*, *Завершен*, *Неудача*.
4. **Администрирование**
   - Для роли `ADMIN`: просмотр пользователей (`/list-users`), назначение/снятие прав (`/add-admin`, `/remove-admin`), просмотр администраторов (`/list-admin`).
   - Получение API-ключа через эндпоинт `auth/` для доступа к HTTP-эндпоинтам.
5. **Справочная информация**
   - Команда `/help` выводит список всех доступных команд.

2 Общие требования

2.1 Управление версиями и сотрудничество

- Весь код должен быть размещен в репозитории GitHub.

- Репозиторий должен содержать файл ‘README.md‘ с подробным руководством по:

    - Настройке проекта локально.

    - Сборке и запуску приложения.

    - Развертыванию приложения с использованием Docker.

    - Взаимодействию с Telegram-ботом.

- Docker-образ приложения должен быть размещён на Docker Hub.

2.2 Целевые артефакты

- Приложение должно создавать исполняемый "fat JAR" как основной артефакт.

- Приложение должно быть развертываемо как Docker-контейнер.

- Использовать инструмент сборки Gradle для управления зависимостями, сборки и упаковки приложения.

3 Функциональные требования

3.1 Аутентификация пользователей

Пользователи идентифицируются по id чата в Telegram, авторизация не требуется.

3.2 Решение форм

1. Получение адреса формы от пользователя:

   - пользователь может написать `/solve short_url`,
   - пользователь может написать `/solve long_url`,
   - пользователь может написать текстовое сообщение содержащее url адрес формы;

   в каждом из этих случаев бот забирает ссылку в обработку и сообщает об этом пользователю.

2. После проверки формы на соответствие требованиям через запрос к Google Forms API, бот сообщает пользователю о результате.

3. Получение ответа от LLM:

   - если LLM не ответила за 30 секунд, то есть ее время ответа привысило лимит, пользователь получает от бота сообщение об этом с указанием номера попытки обращения и общего количества попыток;
   - если LLM ответила пользователь получает сформированный ответ в виде списка из вопросов с ответами. В случае, если в вопросе были варианты ответа, рядом с ними указываются уровни уверенности модели в ответах.

3.2.1 Формат ввода

Помимо допустимых команд, разобранных в разделе «Команды бота», бот также принимает текстовые сообщения, если они содержат ссылку на форму, остальная часть сообщения игнорируется.

Проверка корректности ссылки, еще до запроса к Google Forms API, а также поиск ссылки в текстовом сообщении выполняется с использованием регулярного выражения, приведенного в листинге 1.

Листинг 1. Регулярное выражение задающее ссылки на формы Google Forms

```
^(https?:\/\/)?(www\.)?(docs\.google\.com\/(u\/\d\/)?forms\/d\/(e\/)?[a-zA-Z0-9_-]+(\/viewform)?|forms\.gle\/[a-zA-Z0-9_-]+)(\?.*)?$
```

3.3 История и управление формами

Пользователь имеет возможность:

1. Просматривать историю решённых форм.
   - Команда `/history [период]` выводит список последних запросов пользователя.
   - Параметр `период` может принимать значения: `day` (за день), `week` (за неделю), `month` (за месяц) или `all` (вся история). По умолчанию — последние 10 запросов.
   - Для каждого запроса указываются: дата, название формы, статус (`Ожидает`, `В обработке`, `Завершен`, `Неудача`) и `requestId`.
2. Сохранять формы для быстрого доступа.
   - После успешного решения форма автоматически сохраняется в личном списке пользователя с уникальным `id_формы`.
   - Сохранённую форму можно вызвать позже командами `/solve <id_формы>`.
3. Удалять сохранённые формы.
   - Команда `/remove_form <id_формы>` удаляет форму из личного списка (история решений при этом не удаляется).
4. Просматривать список сохранённых форм.
   - Команда `/myforms` выводит все сохранённые пользователем формы с их идентификаторами и названиями.
5. Повторно решать форму.

- Команда `/rescore <id_формы>` запускает повторную генерацию ответов. Предыдущие ответы перезаписываются в истории.

3.4 Статус и информация

- Запрос статуса обработки: команда `/status <requestId>` возвращает текущий статус запроса:
  - `Ожидает` — запрос принят, ожидает обработки.
  - `В обработке` — идёт генерация ответов LLM.
  - `Завершен` — форма успешно решена, ответы готовы.
  - `Неудача` — произошла ошибка.
- Справка: команда `/help` выводит список всех доступных команд с кратким описанием.

3.5 Администрирование (роль ADMIN)

Пользователи с ролью `ADMIN` имеют дополнительные возможности:

- Просмотр списка пользователей: команда `/list-users` выводит всех зарегистрированных пользователей с их `telegramId`, ролью и датой последней активности.
- Назначение администратора: команда `/add-admin <telegramId>` повышает роль пользователя до `ADMIN`.
- Просмотр администраторов: команда `/list-admin` выводит список всех пользователей с ролью `ADMIN`.
- Снятие прав администратора: команда `/remove-admin <telegramId>` понижает роль пользователя до `USER` (нельзя снять права с последнего администратора).

3.6 Ограничения и лимиты

- Активные запросы: один пользователь не может иметь более 1 запроса в статусе `Ожидает` или `В обработке` одновременно.

- Объём хранимых данных: на одного пользователя выделяется не более 10 МБ.
- Общее количество пользователей: система рассчитана на обслуживание не более 1000 активных пользователей.

3.7 Источник данных для LLM

- Бот использует API стороннего LLM-провайдера — GigaChat.
- При недоступности провайдера возвращается ошибка.

4 Общие требования

4.1 Производительность

1. Максимальное время ответа бота – 300мс, за исключением задержек внешних API.

2. Количество единовременных подключений пользователей до 1000.

4.2 Надежность

Повторные попытки при ошибках доступа к LLM. 3 попытки с интервалом в 30 секунд, если ответ не поступил, или 1 секунда, если была получена ошибка.

4.3 Безопасность

1. Данные подключения к базе данных, а также необходимые ключи для внешних API хранятся в переменных окружения.

2. Ввод пользователя валидируется. По списку команд или, в случае с текстовым сообщением, регулярным выражением на наличие ссылки на форму.

4.4 Роли пользователей

1. Пользователь (User) – любой пользователь, идентифицируется по chat_id:

   - Стандартная роль для любого клиента,
   - стандартные права доступа,
   - отправляет запросы через Telegram бота.

2. Администратор (Admin) – пользователь, который может идентифицировать себе по API ключу:

   - API ключ можно получить на endpoint-е `auth/`, передав параметры pass и login.

- Для регистрации пользователя, как администратора, ему необходимо ввести команду `/verify-admin API-key`.
- Имеет доступ к командам администратора.

4.5 Авторизация

1. Поддержка авторизации через Telegram по Chat ID.
2. Поддержка авторизации через HTTP для API:
   - ключи доступа создаются при запросе на `auth/`, при передаче правильных входных данных, которые хранятся в зашифрованном виде в БД; ключи хранятся 15 минут после создания.
   - предназначена только для пользователей группы Admin.

4.6 Эндпоинты

4.6.1 Эндпоинт healthcheck

- Не требует авторизации по ключу API.
- Должен возвращать статус сервера и список авторов проекта.

4.6.2 Эндпоинт users

- Должен быть доступен только по ключу API, то есть для пользователей группы администраторов.
- Должен предоставлять список всех пользователей.

5 Требования к технологическому стеку

Язык программирования: Java 25.

Фреймворк: Spring 7 (WebFlux, Data MongoDB Reactive, Modulith).

Telegram API: Приложение должно интегрироваться с Telegram API для предоставления функциональности бота.

Брокер сообщений: Kafka.

База данных: MongoDB.

Контейнеризация: Docker, Docker Compose.

Тестирование: JUnit.

6 Требования к документации

Должна быть написана документация для API с использованием Spring Docs.

7 Требования к тестированию

Должны быть реализованы тесты для всех модулей.

8 Руководство по сдаче проекта

8.1 Требования

Предоставить документ с требованиями, который включает:

- Цель и функциональность приложения.

- Ключевые функции Telegram-бота.

- Нефункциональные требования.

- Любые предположения или ограничения.

8.2 Архитектура

Предоставить документ по архитектуре, который включает:

- Высокоуровневый обзор дизайна системы.

- Диаграммы (например, компонентные диаграммы, диаграммы последовательностей или блок-схемы) для иллюстрации архитектуры.

- Обоснование выбранного технологического стека.

- Описание того, как приложение будет строиться, развертываться и запускаться.

8.3 Полный проект

Предоставить полный проект в виде:

- Ссылки на репозиторий GitHub с полной кодовой базой.

- Ссылки на Docker Hub, где опубликован Docker-образ.

- Ссылки или имя пользователя Telegram-бота для тестирования и взаимодействия.

9 Глоссарий

Термин	**Определение**
Telegram	Кроссплатформенный мессенджер с поддержкой облачного хранения сообщений.
Telegram-бот	Автоматизированная программа, работающая внутри Telegram и обрабатывающая команды пользователей.
Chat ID	Уникальный числовой идентификатор чата, группы, канала или диалога с пользователем в Telegram, используемый для аутентификации.
Git	Распределённая система контроля версий для отслеживания изменений в файлах и координации работы над проектом.
GitHub	Платформа для хостинга и совместной разработки проектов с использованием Git.
Репозиторий GitHub	Хранилище кода, где фиксируется весь код проекта и история изменений.
README.md	Файл с инструкциями по настройке, сборке, запуску и развертыванию приложения.
Fat JAR	Исполняемый файл Java, включающий все необходимые зависимости для запуска приложения без дополнительных библиотек.
Docker	Платформа для разработки, доставки и запуска приложений в контейнерах.
Docker-контейнер	Изолированный пакет, содержащий приложение и все его зависимости.
Docker Compose	Инструмент для определения и запуска многоконтейнерных Docker-приложений с помощью YAML-файлов.
Docker Hub	Облачный реестр для хранения и распространения Docker-контейнеров.

Продолжение на следующей странице

Термин	**Определение**
YAML-файл	Текстовый файл в формате YAML, используемый для описания конфигураций.
БД (база данных)	Структурированная система хранения данных для эффективной организации и извлечения информации.
API	Набор инструментов для взаимодействия между программами, позволяющий обмениваться данными и функциональностью.
API-ключ	Уникальный идентификатор для аутентификации и авторизации при доступе к API.
Telegram API	Интерфейс для создания Telegram-ботов и интеграции функций Telegram в приложения.
Фреймворк	Комплекс готовых решений и инструментов, ускоряющих разработку программного обеспечения.
Spring Framework	Платформа для разработки Java-приложений, предоставляющая инструменты для создания веб-приложений и микросервисов.
Reactive Spring (WebFlux)	Модуль Spring Framework для создания реактивных, неблокирующих веб-приложений.
MongoDB	NoSQL база данных, хранящая данные в формате документов.
Брокер сообщений	Система, управляющая обменом сообщений между приложениями в распределённой архитектуре.
Kafka	Распределённый программный брокер сообщений.
Spring JPA	Инструмент Spring Framework для упрощения взаимодействия с реляционными базами данных.

Продолжение на следующей странице

Термин	**Определение**
ORM	Технология преобразования данных между несовместимыми типами систем (объектно-ориентированные языки и реляционные БД).
Spring Modulith	Инструмент для структурирования монолитных приложений в модули.
HTTP	Протокол передачи данных в интернете.
Токен	Уникальная строка данных для аутентификации и авторизации.
Эндпоинт (endpoint)	URL-адрес или точка доступа в веб-сервисе для взаимодействия клиента с сервером.
Healthcheck	Специальный эндпоинт для проверки состояния и работоспособности сервера или приложения.
Логирование	Процесс записи событий и действий в системе для мониторинга и анализа.
Валидация	Проверка данных на соответствие заданным критериям.
Регулярное выражение	Формальный язык для поиска и манипуляций с подстроками в тексте на основе метасимволов.
Инъекция	Атака, при которой вредоносный код вставляется в запросы к базе данных.
Переменная окружения	Значение, используемое в командах и сценариях операционной системы.
Экспоненциальная задержка	Метод повторных попыток с экспоненциальным увеличением времени ожидания.
Аутентификация по схеме Bearer	Схема аутентификации HTTP, использующая токены безопасности (bearer tokens).
Роль пользователя	Категория пользователя с определённым уровнем доступа и правами (USER или ADMIN).
Авторизация	Проверка прав доступа пользователя к определённым ресурсам или действиям.

Продолжение на следующей странице

Термин	**Определение**
GigaChat	Сторонний LLM-провайдер, используемый для генерации ответов на вопросы форм.